Predicting Psychiatric Illness Risk from Neuroimaging–Genetics Associations

Master Thesis – Progress Update

Leon Ackermann

March 12, 2026

Data

Raw Participant-Level Data

Two matrices share matched participant rows ($N_1 \dots N_n$):

Structural MRI ($n \times m$) **Genotype** ($n \times l$)

$$\mathbf{X}_{\text{MRI}} = \begin{pmatrix} \text{MRI}_{1,1} & \cdots & \text{MRI}_{1,m} \\ \vdots & \ddots & \vdots \\ \text{MRI}_{n,1} & \cdots & \text{MRI}_{n,m} \end{pmatrix} \qquad \mathbf{X}_G = \begin{pmatrix} G_{1,1} & \cdots & G_{1,l} \\ \vdots & \ddots & \vdots \\ G_{n,1} & \cdots & G_{n,l} \end{pmatrix}$$

- n = number of participants (European ancestry cohort)
- m = MRI phenotypes (volume, area, thickness, intensity, contrast per brain region)
- l = genotyped SNP variants

GWAS Transformation

For every SNP i and MRI phenotype j, a GWAS collapses the n participants into a single z-score:

$$\text{GWAS}(\mathbf{X}_{G_{*,i}},\ \mathbf{X}_{\text{MRI}_{*,j}}) \longrightarrow z_{i,j}$$

Under a linear model for quantitative traits:

$$y = G_{\text{MRI}_{*,j}}\,\beta^G_{i,j} + X\beta^X + e$$

The z-score is $z_{i,j} = \dfrac{\beta^G_{i,j}}{\text{s.e.}(\beta^G_{i,j})}$.

- $\beta^G_{i,j}$: effect size of SNP i on phenotype j
- X, β^X: covariates (age, sex, PCs...)
- p-value: $\chi^2 = \beta^2 \cdot N$, then inverted

Final Tabular ML Dataset (per illness)

Rows = SNPs, features = MRI z-scores, target = illness z-score.

$$\left[\begin{array}{cccc|c} z_{1,1} & z_{1,2} & \cdots & z_{1,m} & z_{\text{ill},1} \\ z_{2,1} & z_{2,2} & \cdots & z_{2,m} & z_{\text{ill},2} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ z_{l,1} & z_{l,2} & \cdots & z_{l,m} & z_{\text{ill},l} \end{array}\right] \begin{array}{l} \leftarrow G_1 \\ \leftarrow G_2 \\ \vdots \\ \leftarrow G_l \end{array}$$

- After clumping: $l \approx 3{,}500$ SNPs, $m \approx 1{,}000$ MRI features
- SCZ has highest predictive power (largest GWAS sample size)
- Negative median z-score is expected: allele ordering in GWAS is arbitrary

P-Value Clumping & Linkage Disequilibrium

Linkage Disequilibrium (LD)

- Nearby nucleotides are correlated because recombination is less likely at short distances.
- $\mathrm{E}[\hat{\beta}_i] = \beta_i + \sum_{j \neq i} r_{ij}\beta_j$ (LD bias within a block).
- LD blocks are estimated in PLINK
- correlation of chosen SNP with other SNPsof LD block is removed.

Clumping

- Pick the lead SNP per LD block across all illnesses
- P-value threshold (e.g. $p < 10^{-3}$) controls how many SNPs survive
  $\Rightarrow$ dataset size vs. signal quality trade-off.
- Same SNP set selected across illnesses
  $\Rightarrow$ cross-illness comparison is valid.

Label Distribution

Dataset Overview – Illnesses

Illness	Full name	SNPs after clumping	p-value threshold
MDD	Major Depressive Disorder	4,045	$< 10^{-3}$
BIP	Bipolar Disorder	4,720	$< 10^{-3}$
SCZ	Schizophrenia	3,533	$< 10^{-4}$
ADHD	Attention Deficit & Hyperactivity Disorder	3,213	$< 10^{-3}$
ASD	Autism Spectrum Disorder	2,890	$< 10^{-3}$
OCD	Obsessive–Compulsive Disorder	2,027	$< 10^{-3}$

Table 1: Number of SNPs retained per illness after p-value clumping.

- SCZ uses a stricter threshold ($p < 10^{-4}$) because its GWAS has the largest sample size, yielding many significant hits.
- All current experiments target SCZ; other illnesses are planned as extensions.

Goal & Task Formulation

ML Objective

Learn a mapping:

$$f(\mathbf{z}_i) \approx y_i, \qquad \mathbf{z}_i = [z_{i,1}, \ldots, z_{i,m}] \in \mathbb{R}^m, \quad y_i \in \mathbb{R}$$

Two formulations:

1. **Regression**: predict the continuous illness z-score directly.
   $f : \mathbb{R}^m \to \mathbb{R}$
2. **Binary classification**: threshold the z-score $\Rightarrow$ risk / no-risk label.
   Potentially more feasible given label distribution.

Both are instances of *tabular machine learning*, which remains notoriously hard for unstructured feature sets [1], [2].

Methods

Model Landscape

Baselines

- Linear Regression
- Lasso (ℓ_1)
- Ridge (ℓ_2)
- ElasticNet

Boosted Trees

- XGBoost [3]
- LightGBM [4]
- CatBoost [5]

Deep Learning

- ResidualDNN *(ours)*
- MLP + Reg. Cocktail [1]
- TabNet [6]
- TabPFNv2 [7]

Evaluation Protocol

All models: 10 random seeds, 80/20 train/test. Metrics: R^2, MSE (regression); AUC-ROC (classification).

Baseline Hyperparameters

Model	Key Hyperparameter	Value
Linear Regression	–	OLS (no regularisation)
Lasso	α	0.0212 (via LassoCV)
Ridge	α	1000.0 (via RidgeCV)
ElasticNet	α, ℓ_1-ratio	0.0695, 0.1
XGBoost	`n_estimators`	200
	`max_depth`	3
	`learning_rate`	0.1
	`subsample`	0.9
	`colsample_bytree`	0.8

Table 2: Key hyperparameters for baselines, selected via cross-validated grid search.

All models: 80/20 train/test split, 10 seeds (42–51).
Regularisation hyperparameters selected via cross-validated grid search.

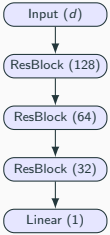
ResidualDNN Architecture

Architecture

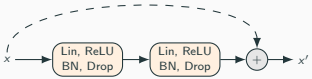
- Hidden dims: $[128, 128, 64, 64, 32, 32]$
- 3 **residual blocks** with additive skip connections

Training

- Optimiser: Adam, lr $= 10^{-5}$, weight decay $= 10^{-5}$, gradient clipping ≤ 1.0
- 300 epochs, batch size 32, Early Stopping (patience 20)
- LR schedule: Cosine Annealing with warm restarts (T_0=15, T_{mult}=2)



ResBlock detail:



Outlook: The *Regularisation Cocktail* [1] adds more implicit, structural, data augmentation, and ensemble methods on top of a plain MLP—further avenues for improvement.

TabPFN – Bayesian View and PFN Objective

Posterior predictive distribution (PPD): for a test sample x_{test}, the model predicts $p(\cdot \mid x_{\text{test}}, D_{\text{train}})$ conditioned on $D_{\text{train}} := \{(x_1, y_1), \ldots, (x_n, y_n)\}$.

The Bayesian PPD integrates over hypotheses $\phi \in \Phi$:

$$p(y \mid x, D) \propto \int_{\Phi} p(y \mid x, \phi)\, p(D \mid \phi)\, p(\phi)\, d\phi, \qquad \phi \in \Phi$$

TabPFN approximates this posterior predictive with a transformer q_θ trained via

$$\mathcal{L}_{\text{PFN}} = \mathbb{E}_{(\{(x_{\text{test}}, y_{\text{test}})\} \cup D_{\text{train}}) \sim p(D)} \left[-\log q_\theta\big(y_{\text{test}} \mid x_{\text{test}}, D_{\text{train}}\big) \right].$$

Data is sampled from the prior predictive process:

$$p(D) = \mathbb{E}_{\phi \sim p(\phi)}\big[p(D \mid \phi)\big], \qquad \phi \sim p(\phi),\ D \sim p(D \mid \phi).$$

At inference, we input $\langle D_{\text{train}}, x_{\text{test}} \rangle$ and obtain

$$q_\theta\left(y \mid x_{\text{test}}, D_{\text{train}}\right)$$

as a single forward-pass approximation of the PPD. Further information can be found in [8].

TabPFNv2 - Training and Prior

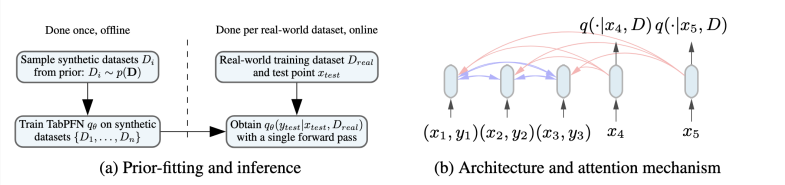


Figure 1: Training Overview [8]

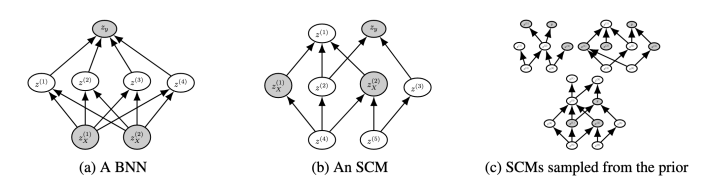


Figure 2: Prior Distribution [8]

TabPFNv2 - Architecture

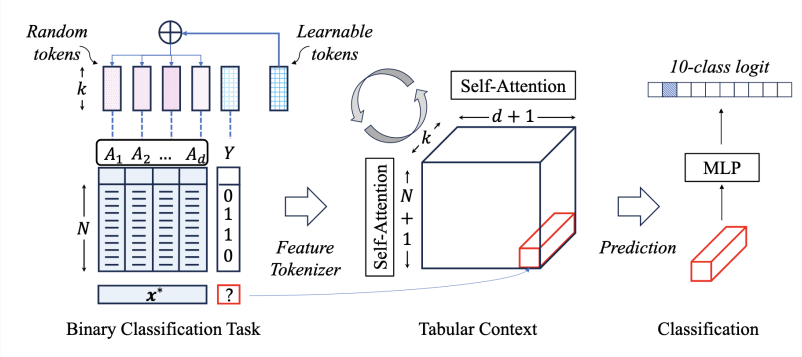


Figure 3: Architecture [9]

- Enables **in-context learning**: train samples are provided as context at inference.
- Approximates Bayesian inference by directly modeling the posterior predictive distribution (PPD).

Results

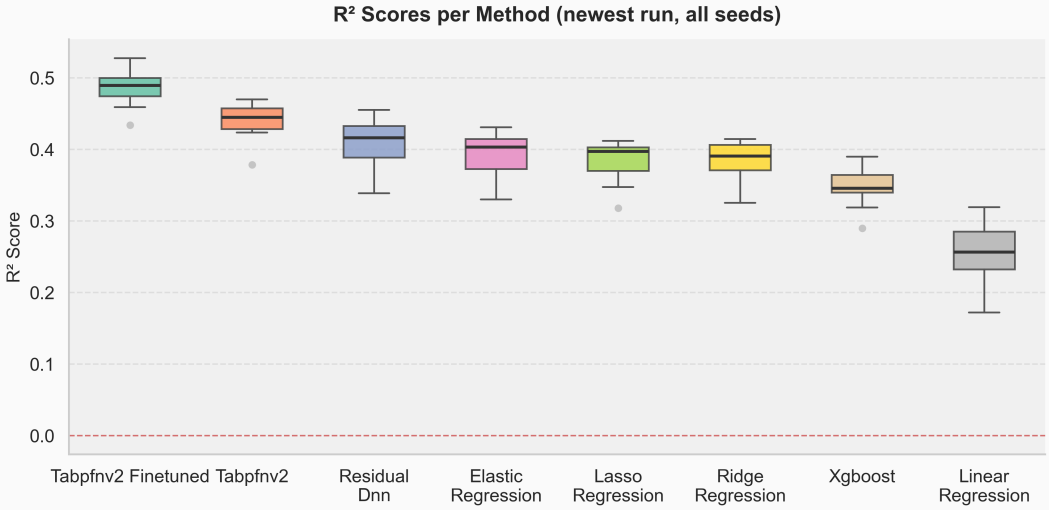


Figure 4: Boxplot of R^2 across models, aggregated over 10 seeds (42–51), 80/20 split.

Results – Schizophrenia (SCZ)

Model	Mean R^2	Mean MSE
TabPFNv2 finetune	**0.48**±0.025	**11.42**
TabPFNv2	0.44±0.026	12.49
Residual DNN	0.409±0.035	13.10
ElasticNet	0.393±0.031	13.47
Ridge	0.385±0.027	13.64
Lasso	0.382±0.030	13.70
XGBoost	0.349±0.029	14.45
Linear Regression	0.258±0.046	16.47

Table 3: Aggregated over 10 seeds (42–51), 80/20 split.

- Regularised linear models outperform plain OLS substantially (+0.12 R^2).
- XGBoost under-performs linear baselines—likely due to limited sample size (∼3.5k).
- Neural networks are slighly better than linear models, suggesting some non-linear signal is present?
- TabPFNv2 achieves the best R^2 so far, suggesting non-linear signal is present.

Next Steps

Next Steps

1. **Data distribution**
   Improve label distribution, represent whole range of z-scores
2. **Regularisation Cocktail MLP**
   Systematically search implicit, structural, data augmentation, and ensemble methods [1]. Will take a while to train, no guarantee of improvement.
3. **TabPFNv2 optimization**
   Hyperoptimization, fine-tuning and continued pre-training for TabPFNv2. Inspired by [7]. Promising avenue to boost performance.
4. **TabPFNv2 Explainability**
   Use SHAP values to identify which MRI features drive predictions; compare with known biology. Create causal graphs of feature importance.
5. **Graph Neural Network**
   Treat brain regions as nodes with MRI features; edge definition TBD (spatial adjacency, functional connectivity, or learned).

References

[1] A. Kadra, M. Lindauer, F. Hutter, and J. Grabocka, **"Well-tuned Simple Nets Excel on Tabular Datasets,"** en,

[2] D. Holzmüller, L. Grinsztajn, and I. Steinwart, **Better by default: Strong pre-tuned MLPs and boosted trees on tabular data,** en, 2024. DOI: 10.18419/DARUS-4555. [Online]. Available: https://darus.uni-stuttgart.de/citation?persistentId=doi:10.18419/darus-4555 (visited on 03/06/2026).

References ii

[3] T. Chen and C. Guestrin, **"XGBoost: A Scalable Tree Boosting System,"** en, in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, San Francisco California USA: ACM, Aug. 2016, pp. 785–794, ISBN: 978-1-4503-4232-2. DOI: 10.1145/2939672.2939785. [Online]. Available: https://dl.acm.org/doi/10.1145/2939672.2939785 (visited on 03/09/2026).

[4] G. Ke, Q. Meng, T. Finley, *et al.*, **"LightGBM: A Highly Efficient Gradient Boosting Decision Tree,"** en,

[5] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, **"CatBoost: Unbiased boosting with categorical features,"** en,

[6] S. Ö. Arik and T. Pfister, **"TabNet: Attentive Interpretable Tabular Learning,"** en, *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 8, pp. 6679–6687, May 2021, ISSN: 2374-3468. DOI: 10.1609/aaai.v35i8.16826. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/16826 (visited on 03/05/2026).

References iii

[7] N. Hollmann, S. Müller, L. Purucker, *et al.*, **"Accurate predictions on small data with a tabular foundation model,"** en, *Nature*, vol. 637, no. 8045, pp. 319–326, Jan. 2025, ISSN: 1476-4687. DOI: 10.1038/s41586-024-08328-6. [Online]. Available: https://www.nature.com/articles/s41586-024-08328-6 (visited on 03/06/2026).

[8] N. Hollmann, S. Müller, K. Eggensperger, and F. Hutter, **TabPFN: A Transformer That Solves Small Tabular Classification Problems in a Second,** en, arXiv:2207.01848 [cs], Sep. 2023. DOI: 10.48550/arXiv.2207.01848. [Online]. Available: http://arxiv.org/abs/2207.01848 (visited on 03/09/2026).

[9] H.-J. Ye, S.-Y. Liu, and W.-L. Chao, **A Closer Look at TabPFN v2: Understanding Its Strengths and Extending Its Capabilities,** en, arXiv:2502.17361 [cs], Jun. 2025. DOI: 10.48550/arXiv.2502.17361. [Online]. Available: http://arxiv.org/abs/2502.17361 (visited on 03/09/2026).